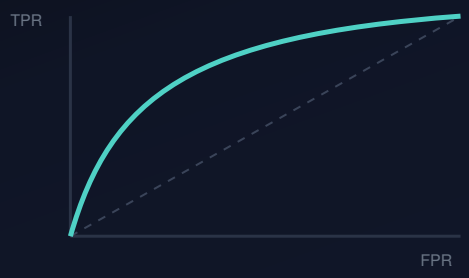
MACHINE LEARNING · CLASSIFICATION METRICS

The ROC Curve From Basics to Pro

A complete, from-scratch guide to understanding thresholds, confusion matrices, True Positive Rate, False Positive Rate, the ROC curve, and AUC — built up step by step with visual intuition, worked examples, and Python code.

Binary Classification | Confusion Matrix | TPR & FPR | Threshold Tuning | AUC & Model Comparison



Adapted & restructured tutorial · dark-mode reading edition

Table of Contents

Follow the sections in order — each one builds on the last.

01 Why does the ROC curve even exist?

02 Recap: Regression vs. Classification

— Binary classification and how a model is trained/tested

03 Probabilities and the Idea of a Threshold

04 The Two Kinds of Mistakes a Classifier Makes

— Why one mistake can be far more costly than the other

05 The Confusion Matrix (Full Recap)

06 True Positive Rate (TPR) — the "Benefit"

07 False Positive Rate (FPR) — the "Cost"

08 Building the ROC Curve, Step by Step

— Why TPR and FPR do NOT move together linearly

09 Reading the ROC Curve & Picking the Best Threshold

10 Hands-on: ROC Curve in Python (Diabetes Dataset)

11 AUC — Area Under the Curve

12 Comparing Two Models with ROC & AUC

13 Summary Cheat-Sheet & Glossary

SECTION 01

Why Does the ROC Curve Even Exist?

Before learning what a tool **is**, it helps to first ask why it was needed in the first place. So before we define the ROC curve mathematically, let's understand the problem it was built to solve.

Almost every classification model doesn't just spit out a plain "yes" or "no". Internally, most algorithms (logistic regression, decision trees, SVMs, neural networks) first compute a **probability** — a number between 0 and 1 — and only then convert that probability into a final class label using a cut-off point called a **threshold**.

> **CORE IDEA**
>
> The threshold you pick changes your model's behaviour completely — even though the underlying model never changed. A different threshold means a different balance between two kinds of mistakes. The ROC curve exists to help you **see and choose** that balance, and to **compare** two different models fairly, regardless of the threshold each one might use.

So there are two big reasons we study the ROC curve:

1. **Threshold selection** — finding the cut-off probability that gives the best trade-off for your specific problem.
2. **Model comparison** — comparing two entirely different models (e.g. Logistic Regression vs. SVM) on equal footing, independent of any one threshold.

Keep these two goals in mind — everything in this guide eventually serves one of them.

SECTION 02

Recap: Regression vs. Classification

In supervised machine learning, every problem falls into one of two buckets, depending on what kind of output column (target) you are trying to predict.

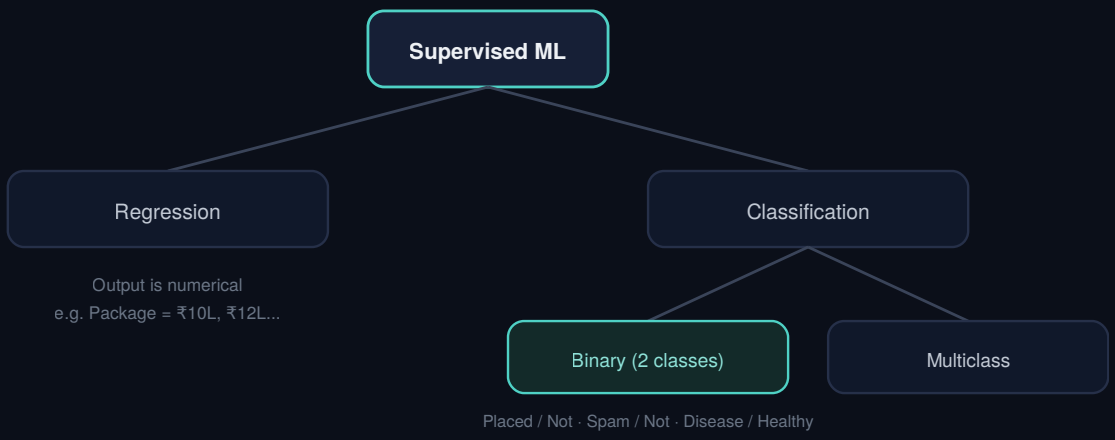


Fig 2.1 — Where classification (and specifically binary classification) sits inside supervised learning. The ROC curve is built for binary classification problems.

Regression — output is a number (salary, price, temperature)

Classification — output is a category (placed / not placed)

Binary classification — exactly two classes (yes / no)

Common binary classification examples: will a student get **placed or not**, is an email **spam or not**, does a patient have **heart disease or not**. The ROC curve is used almost exclusively on this kind of two-class problem.

How a Model is Actually Trained and Tested

Say we have data for 1,000 students (GPA, IQ, and whether they were placed). Before we can evaluate anything, the workflow always follows the same pattern:

1. **Split the data.** Divide it into a training set (e.g. 800 students) and a test set (e.g. 200 students) that the model has never seen.
2. **Train.** Fit any classification algorithm — Logistic Regression, Decision Tree, Naive Bayes, SVM, or a Neural Network — on the training set.
3. **Predict.** Use the trained model to make predictions on the unseen 200 test students.

4. **Evaluate.** Compare those predictions against the real, known outcomes to see how well the model actually performed.

IMPORTANT DETAIL

When an algorithm makes a prediction on test data, it does **not** directly output a clean 0 or 1. It outputs a **probability** — a number between 0 and 1 — representing how likely it is that the student belongs to the positive class (e.g. "placed"). Turning that probability into a final 0/1 label is where the threshold comes in, and that's exactly what the next section covers.

SECTION 03

Probabilities and the Idea of a Threshold

Suppose the model says a particular student has a **45% probability** of being placed. Is that a "yes" or a "no"? We need a rule to decide — and that rule is the **threshold**.

The most common default threshold is **0.5**: any probability ≥ 0.5 is labelled positive (e.g. "placed" / "spam"), and anything below 0.5 is labelled negative.

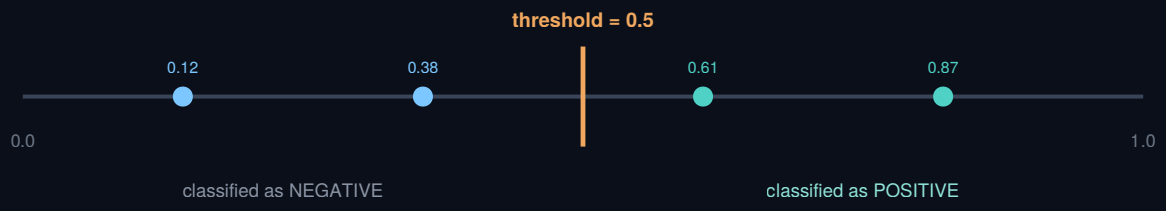


Fig 3.1 — Each dot is one student's predicted probability of being placed. The threshold decides where the cut-off line falls.

Here's the key insight the rest of this guide is built on: **the threshold is a dial you can turn.** Moving it changes nothing about the trained model itself — only how its raw probabilities get converted into final labels.

SECTION 04

The Two Kinds of Mistakes a Classifier Makes

In any binary classification problem, there are exactly two distinct ways to be wrong:

MISTAKE TYPE 1

Actual = Positive, Predicted = Negative. Example: the student *was* actually placed, but the model predicted "not placed". Example: the email genuinely *was* spam, but the model let it through.

MISTAKE TYPE 2

Actual = Negative, Predicted = Positive. Example: the student was *not* placed, but the model predicted "placed". Example: the email was *not* spam, but the model flagged it as spam.

Both mistakes are not always equally bad

Consider the spam-filter example carefully:

THE DANGEROUS MISTAKE

A real placement-offer email — telling a student to report to office on a specific date — gets wrongly marked as spam and buried. The student never sees it. This is a **false positive** for "spam", and it is genuinely damaging.

THE TOLERABLE MISTAKE

A spam/promotional email slips through and lands in the inbox instead of the spam folder. Mildly annoying, but essentially harmless — the student just sees one extra ad.

Because one error is clearly riskier than the other here, you would want to make your model more *cautious* about calling something spam. The lever for that caution is exactly the threshold: instead of the default 0.5, you could raise it to 0.75 or even 0.9, so that only very high-confidence predictions get labelled "spam". This is called **threshold selection**, and the hard part is this —

THE REAL PROBLEM

You don't know in advance which threshold is correct. Is it 0.35? Is it 0.5? Is it 0.65 or 0.85? There's no way to answer this just by staring at one confusion matrix. You need to see **how the trade-off behaves across every possible threshold at once** — and that visual is precisely the ROC curve.

SECTION 05

The Confusion Matrix (Full Recap)

The confusion matrix is a simple "report card" for a binary classifier: rows are the **actual** values, columns are the **predicted** values.

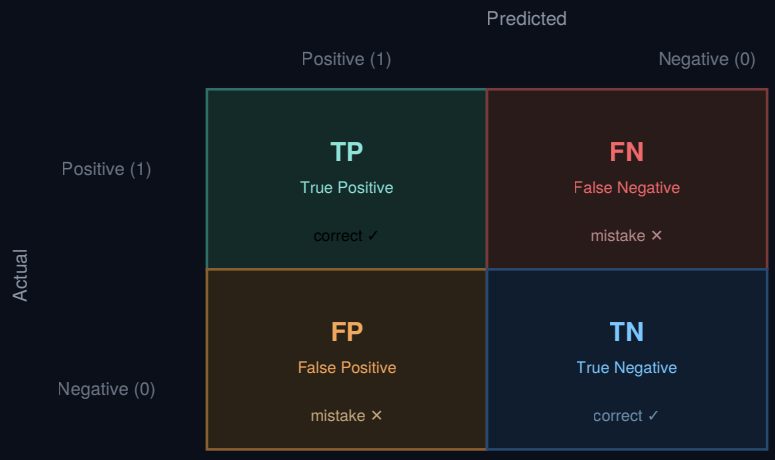


Fig 5.1 — The four possible outcomes of a binary classifier. Green/blue cells = correct. Red/orange cells = the two kinds of mistakes from Section 4.

Cell	Meaning	Example (placement problem)
TP — True Positive	Actual = Yes, Predicted = Yes	Student was placed, model correctly said "placed"
TN — True Negative	Actual = No, Predicted = No	Student wasn't placed, model correctly said "not placed"
FP — False Positive	Actual = No, Predicted = Yes	Student wasn't placed, model wrongly said "placed"
FN — False Negative	Actual = Yes, Predicted = No	Student was placed, model wrongly said "not placed"

WORKED EXAMPLE — 200 TEST STUDENTS

Suppose out of 200 students in the test set, we get: **TP = 100**, **TN = 80**, **FP = 5**, **FN = 15**. Correct predictions = 100 + 80 = **180**. Mistakes = 5 + 15 = **20**. Together these four numbers always add up to the total size of the test set (here, 200).

This matrix is the raw material for everything that follows. From these four numbers, we can compute two very important rates — **TPR** and **FPR** — for *any* threshold. Those two rates are the X and Y axes of the ROC curve.

SECTION 06

True Positive Rate (TPR) — the "Benefit"

TRUE POSITIVE RATE (A.K.A. SENSITIVITY / RECALL)

$$TPR = \frac{TP}{TP + FN}$$

Out of all the ACTUAL positives, what fraction did the model correctly catch?

SPAM EXAMPLE

Say your test set has 100 real spam emails. Your model correctly labels 80 of them as spam, and misses 20 (calls them "not spam"). TPR = 80 / 100 = **80%**. In plain words: out of every 100 spam emails that exist, you successfully caught 80 of them.

Think of TPR as the **benefit** you're getting from the model — the whole point of building a spam filter is to catch spam, and TPR tells you what percentage of that job is actually getting done.

- **TPR = 1 (100%)** → the model caught every single actual positive. Full benefit.
- **TPR = 0 (0%)** → the model caught none of the actual positives (all of them were mislabelled as negative). Worst case.
- **Higher TPR is always better** — you always want to maximize it.

SECTION 07

False Positive Rate (FPR) — the "Cost"

FALSE POSITIVE RATE

$$FPR = \frac{FP}{FP + TN}$$

Out of all the ACTUAL negatives, what fraction did the model wrongly flag as positive?

CUSTOMER CHURN EXAMPLE

Suppose the model flags 10 customers as "about to leave", but in reality none of them were leaving. To retain them you give each one a ₹100/month discount — that's ₹1,000 spent unnecessarily. This is the **cost** of false positives: real money, or real trust, spent chasing mistakes.

FPR tells you how **expensive** your model is — how much collateral damage (wrongly bothering good customers, burying important emails, mislabelling healthy patients as sick) it causes along the way.

- **Lower FPR is always better** — you always want to minimize it.
- **FPR = 0** → the model never wrongly flags a true negative. Zero cost.
- **FPR = 1** → the model wrongly flags every single true negative. Maximum cost.

THE TRADE-OFF, IN ONE LINE

TPR is the benefit you want to maximize. FPR is the cost you want to minimize. Every threshold you choose lands you at some (FPR, TPR) point — and the ROC curve is simply the map of every possible point you could land on.

SECTION 08

Building the ROC Curve, Step by Step

ROC stands for *Receiver Operating Characteristic* — an old radar-engineering term you never really need to remember. What matters is the definition:

> **DEFINITION**
>
> The ROC curve is a plot of **TPR (y-axis)** against **FPR (x-axis)**, traced out as you sweep the classification threshold across every value from 0 to 1.

The recipe

1. **Train once.** Train your model (e.g. Logistic Regression) on the training set, exactly as before.
2. **Get probabilities.** On the test set, get the predicted probability for every row (not the hard 0/1 label — the raw probability).
3. **Sweep the threshold.** Pick many threshold values between 0 and 1 (e.g. 0.01, 0.02, ... 0.99).
4. **Build a confusion matrix per threshold.** For each threshold, convert probabilities → labels, then compute TP, TN, FP, FN.
5. **Compute TPR and FPR.** For each threshold's confusion matrix, calculate TPR and FPR using the formulas from Sections 6 and 7.
6. **Plot.** Each threshold gives one (FPR, TPR) point. Plotting all of them, in order, traces out the ROC curve.

What happens at the extremes?

> **VERY LOW THRESHOLD (≈ 0.05)**
>
> You call almost everything "positive". Nearly every real positive gets caught, so **TPR → 1**. But you also wrongly flag most real negatives, so **FPR → 1** too. You land near the top-right corner (1, 1).

> **VERY HIGH THRESHOLD (≈ 0.95)**
>
> You only call something "positive" when the model is extremely confident. Very few real positives get through, so **TPR → 0**. But you also almost never wrongly flag a negative, so **FPR → 0** too. You land near the bottom-left corner (0, 0).

Why isn't the relationship a straight line?

It's tempting to assume that as you relax the threshold, TPR and FPR increase in a simple 1-to-1, linear way. This is a common misconception — the real relationship is curved. Here's the geometric reasoning:

Near the high-threshold end, probabilities that cross the line first tend to belong mostly to *true* positives (the model was already very confident about them). So as you lower the threshold slightly, **TP climbs quickly while FP barely moves** — TPR rises fast, FPR stays flat. The curve shoots almost straight up.

As you keep lowering the threshold into the middle range, you start pulling in the "borderline" cases — probabilities around 0.4–0.6 — which are a mix of true and false positives. Now **both TP and FP grow together**, so the curve bends and starts moving right as well as up.

Finally, near the low-threshold end, you're forced to accept almost everything, including plenty of genuine negatives. Now **FP grows quickly while TP has little room left to grow** (it's already near its max) — the curve flattens out and moves mostly sideways to reach (1, 1).

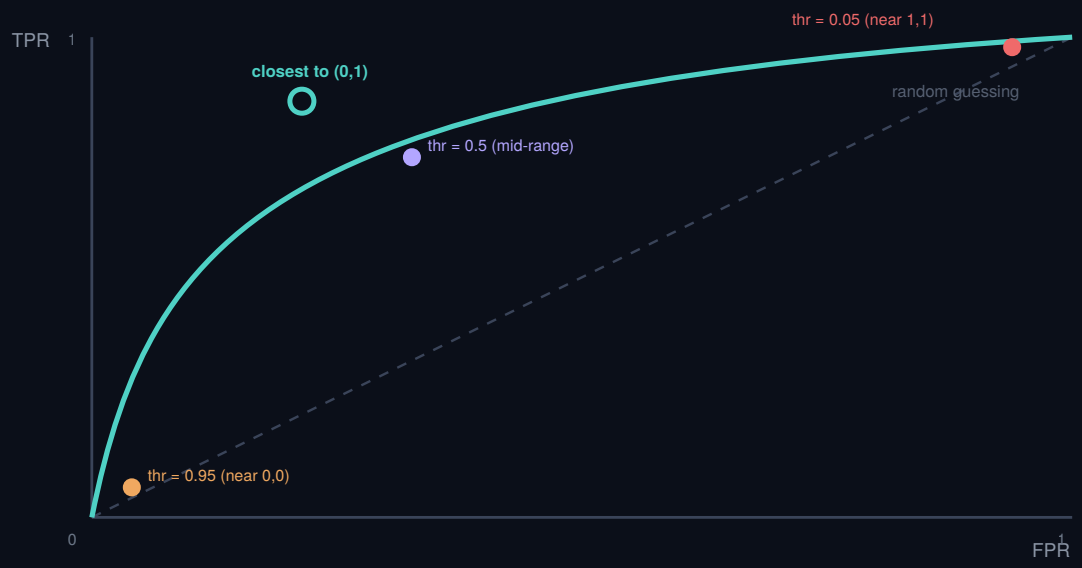


Fig 8.1 — The ROC curve rises steeply first (TP climbs fast, FP stays low), then bends and flattens as FP starts climbing too. It is never a straight line.

SECTION 09

Reading the ROC Curve & Picking the Best Threshold

The reference diagonal

The dashed diagonal line from (0,0) to (1,1) represents a model that is **guessing randomly** — for every extra bit of benefit (TPR) it gains, it pays an equal amount of cost (FPR). Any useful model's curve should bulge well above this diagonal. The closer your curve hugs the top-left corner, the better the model.

The ideal point

The single best possible outcome is the top-left corner: **(FPR = 0, TPR = 1)** — you catch 100% of the actual positives while wrongly flagging 0% of the actual negatives. Real models can rarely reach this exactly, but it's the target we measure distance against.

> **PRACTICAL RULE FOR PICKING A THRESHOLD**
>
> Walk along the curve and find the point that is either **closest to the top-left corner (0, 1)**, or equivalently, **farthest above the diagonal** (this second version is known as Youden's J statistic: J = TPR − FPR). The threshold that produced that point is your "best", balanced choice.

Of course, "best" is context-dependent. If false negatives are far more dangerous in your problem (e.g. missing a genuine disease case), you may deliberately choose a threshold further down the curve that favours a higher TPR even at the cost of a higher FPR — and vice versa. The ROC curve doesn't make that judgement call for you; it just shows you every option so you can make an informed decision instead of guessing.

SECTION 10

Hands-on: ROC Curve in Python (Diabetes Dataset)

Let's make this concrete with the classic diabetes dataset, where the goal is to predict whether a patient is diabetic based on features like glucose level, BMI, and number of pregnancies.

```
# 1. Imports
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# 2. Load data and split into train / test
df = pd.read_csv("diabetes.csv")
X = df.drop(columns=["Outcome"])
y = df["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Train a logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# 4. Get PROBABILITIES, not hard labels
y_probs = model.predict_proba(X_test)[:, 1]

# 5. roc_curve sweeps every threshold automatically
#    and returns matching FPR / TPR / threshold arrays
fpr, tpr, thresholds = roc_curve(y_test, y_probs)

# 6. Plot the ROC curve
plt.plot(fpr, tpr, label="Logistic Regression")
plt.plot([0, 1], [0, 1], linestyle="--", label="Random guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

# 7. Find the best threshold: farthest above the diagonal
best_idx = (tpr - fpr).argmax()
best_threshold = thresholds[best_idx]
print("Best threshold:", best_threshold)
```

WHAT ROC_CURVE() DOES INTERNALLY

It repeats the manual process from Section 8 automatically: for a large number of threshold values, it builds a confusion matrix, computes TPR and FPR, and returns all of them as arrays — ready to plot directly.

NOTE ON SMOOTHNESS

If you only test a handful of threshold values (say 40 values by hand), the curve will look jagged and star-shaped. Using every unique probability value as a threshold (which is what `roc_curve()` does by default) produces a smooth curve.

SECTION 11

AUC — Area Under the Curve

Once you have a full ROC curve, you can boil it down to a single number: the **area under that curve**, ranging from the (0,0) corner to the (1,1) corner. This single number is called **AUC** (Area Under the ROC Curve), sometimes written AUC-ROC.

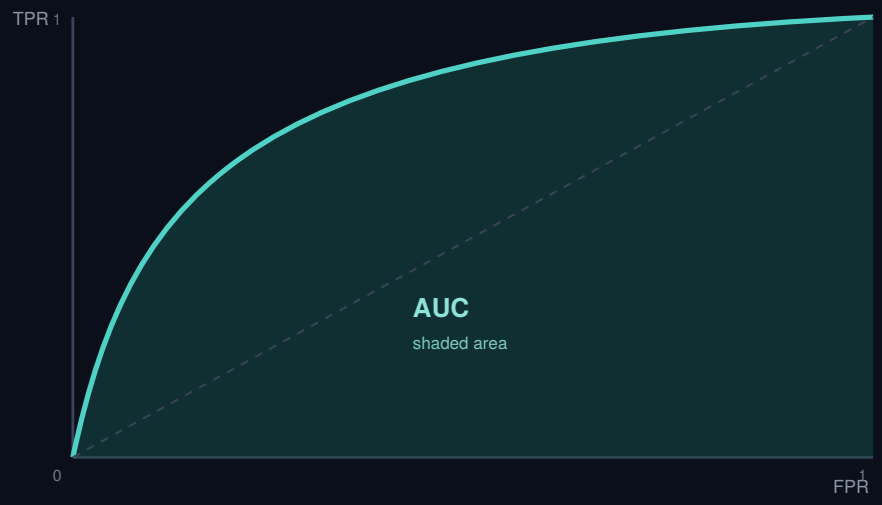


Fig 11.1 — AUC is simply the shaded region under the ROC curve. It summarizes performance across every possible threshold in one number.

AUC RANGE AND MEANING

$$0 \leq AUC \leq 1$$

AUC value	Meaning
1.0	Perfect model — perfectly separates positive and negative classes at some threshold.
0.5	No better than random guessing (this is exactly the area under the diagonal line).
0.0	Perfectly wrong — the model is systematically labelling positives as negatives and vice versa.
Higher = better	In general, higher AUC values indicate better overall classification performance.

WHY AUC IS USEFUL

AUC gives you an **aggregate measure of performance across all possible classification thresholds** — instead of judging a model at just one threshold (like 0.5), you get a single score that reflects how well it performs overall, everywhere along the curve.

SECTION 12

Comparing Two Models with ROC & AUC

This is the second big reason ROC/AUC exists (Section 1): comparing entirely different algorithms fairly, without committing to a single threshold up front.

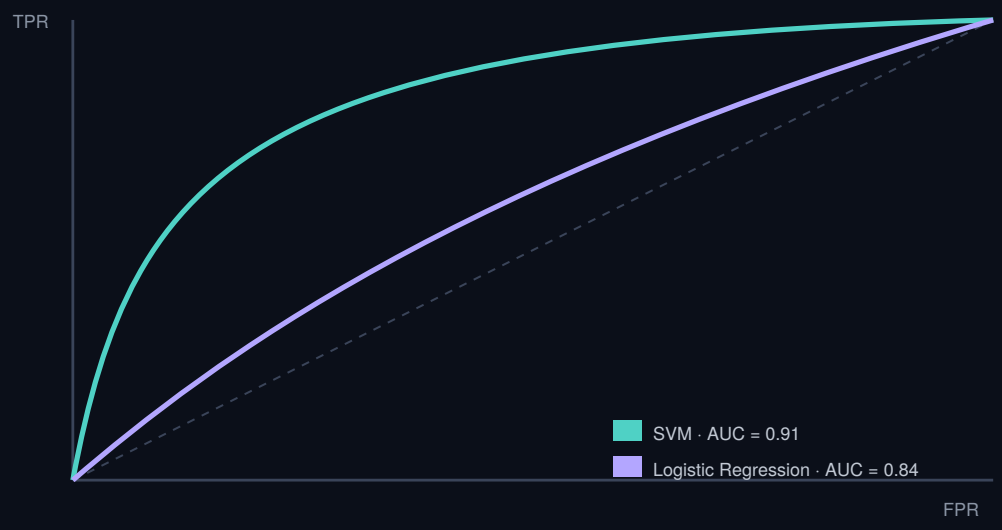


Fig 12.1 — Two models' ROC curves plotted together. The curve that bulges closer to the top-left corner (here, SVM) has the higher AUC and is the better classifier overall.

READING THE COMPARISON

Whichever curve sits consistently above and to the left of the other one is the stronger model — it achieves a higher TPR for the same FPR (or equivalently, a lower FPR for the same TPR) across the board. In the figure above, the model with **AUC = 0.91** outperforms the one with **AUC = 0.84**, so between these two, the higher-AUC model is preferred.

CAVEAT

A single train/test split can be noisy — the "winning" model can flip depending on how the data was split. In practice, proper cross-validation (averaging AUC across multiple folds) gives a much more reliable comparison than a one-off split.

SECTION 13

Summary Cheat-Sheet & Glossary

The big picture, in five lines

1. A classifier outputs a **probability**, and a **threshold** turns it into a 0/1 label.
2. Every threshold produces its own **confusion matrix** (TP, TN, FP, FN).
3. From that matrix we compute **TPR** (benefit, maximize) and **FPR** (cost, minimize).
4. Plotting TPR vs. FPR across every threshold traces the **ROC curve**.
5. The area under that curve, **AUC**, is a single number summarizing overall model quality — useful for both threshold selection and model comparison.

Formula reference

Term	Formula	Meaning
TPR (Recall / Sensitivity)	TP / (TP + FN)	% of actual positives correctly caught
FPR	FP / (FP + TN)	% of actual negatives wrongly flagged
Youden's J	TPR − FPR	used to pick the best balanced threshold
AUC	∫ TPR d(FPR) over [0,1]	area under the ROC curve, 0 to 1

Glossary

Threshold — cut-off probability that decides the predicted class

Confusion Matrix — 2×2 table of TP / TN / FP / FN

True Positive Rate — see Section 6

False Positive Rate — see Section 7

ROC Curve — TPR vs FPR across all thresholds

AUC — area under the ROC curve

Random classifier — the diagonal line, AUC = 0.5

End of tutorial — from "why does this exist" all the way to comparing models with AUC. Revisit the ROC curve diagram in Section 8 whenever the shape stops making intuitive sense; everything else in this guide builds on that one picture.